arXiv:2509.09689v1 [cs.IR] 18 Aug 2025

Personas within Parameters: Fine-Tuning Small Language Models with Low-Rank Adapters to Mimic User Behaviors

Himanshu Thakur
JPMorgan Chase & Co
Palo Alto, CA 94304
himanshu.thakur@chase.com

Eshani Agrawal
JPMorgan Chase & Co
Palo Alto, CA 94304
eshani.agrawal@jpmchase.com

Smruthi Mukund
JPMorgan Chase & Co
Palo Alto, CA 94304
smruthi.mukund@chase.com

Abstract

A long-standing challenge in developing accurate recommendation models is simulating user behavior, mainly due to the complex and stochastic nature of user interactions. Towards this, one promising line of work has been the use of Large Language Models (LLMs) for simulating user behavior. However, aligning these general-purpose large pre-trained models with user preferences necessitates: (i) effectively and continously parsing large-scale tabular user-item interaction data, (ii) overcoming pre-training-induced inductive biases to accurately learn user specific knowledge, and (iii) achieving the former two at scale for millions of users. While most previous works have focused on complex methods to prompt an LLM or fine-tune it on tabular interaction datasets, our approach shifts the focus to extracting robust textual user representations using a frozen LLM and simulating cost-effective, resource-efficient user agents powered by fine-tuned Small Language Models (SLMs). Further, we showcase a method for training multiple low-rank adapters for groups of users or *persona*, striking an optimal balance between scalability and performance of user behavior agents. Our experiments provide compelling empirical evidence of the efficacy of our methods, demonstrating that user agents developed using our approach have the potential to bridge the gap between offline metrics and real-world performance of recommender systems.

1 Introduction

In the rapidly evolving landscape of recommendation systems, user behavior simulation has become crucial for cost, safety, and efficiency benefits. A promising line of work towards this has been Agentic-LLM systems to simulate a user in a recommendation environment. While prior work has been able to effectively establish the value of such systems, representing a user faithfully and at scale has for a large part been unsolved. Thus, we posit that developing robust user simulation agents requires the ability to continuously capture a user's static and dynamic behavior from large volumes of noisy interactions.

To address these challenges, we propose a method to build scalable and efficient user behavior agents. First, we transform large volumes of user interactions into meaningful textual representations. Next, we use low-rank adapters to fine-tune a SLM on this data, creating a user agent that closely

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

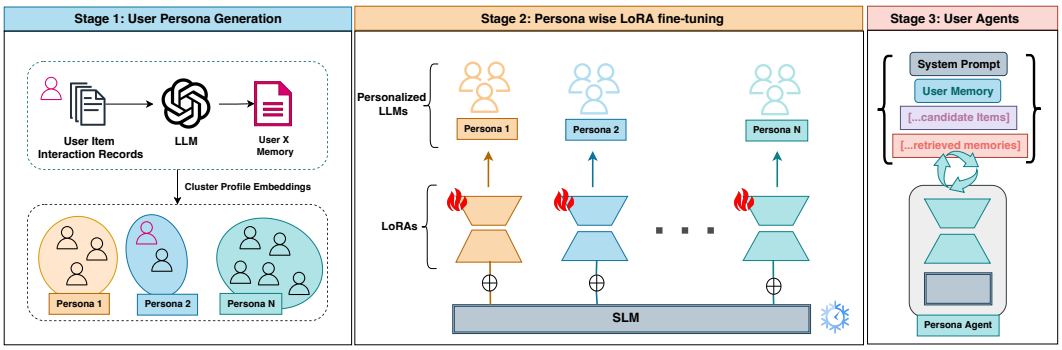

Figure 1: An overview of our proposed approach. First, we iterate over all user-item interaction records to generate user profile features and explanations for certain unique interactions. Then, we cluster users based on their profile embedding to generate N personas. In stage 2, we train a low-rank adapter on all user interaction data within the persona keeping the base SLM weights frozen. Finally, in stage 3, we utilize at persona-level SLMs to build user agents.

mimics actual user actions. While previous works have employed Retrieval Augmented Generation (RAG) systems to build agents, we argue that fine-tuning an SLM is more effective and scalable in real-world applications. Additionally, we show that users can be grouped into personas, balancing the number of parameters required with the personalization quality of these user agents. Thus, our main contributions can be summarized as follows:

1. We demonstrate a hierarchical knowledge distillation process that utilizes an LLM and self-reflection to convert tabular user interactions into textual user profiles and ground their interactions in reason. We find that both of these components are key to improving the user agent's performance.
2. We show that low-rank adaptation of SLMs can match or exceed the performance of frozen LLMs for building personalized agents. Furthermore, we show that Retrieval Augmented Fine-tuning (RAFT) helps in the effective utilization of short-term and long-term memory needed to build the user agent.
3. Finally, we show that clustering users based on their profiles generated during distillation helps strike a balance between personalization performance and the number of model parameters needed.

2 Related Works

In recent years, the field of LLMs has witnessed significant advancements, particularly in the domains of personalization, simulation of human behavior, and recommendation systems. This section reviews the most pertinent works, categorized by their thematic focus.

Simulating User Agents using LLMs: The simulation of human behavior through LLMs shows promise for interactive systems. Jiang et al. [4] and Park et al. [10] enhance virtual agents with personality traits and lifelike interactions. Wang et al. [15] and Yang et al. [16] improve recommendation accuracy and adaptability, highlighting LLMs' potential in enhancing user interactions and recommendations. Our works aims to progress this research for SLMs.

Enriching Users and Items using LLMs: Sun et al. [13] propose a framework to enhance user representation by distilling interactions into a hierarchy of user descriptions. This method leverages persona-specific data to improve the contextual appropriateness of response predictions. Zhang et al. [17] explore the use of generative agents in recommendation systems, treating items as agents that engage in multi-turn interactions with users, thereby enriching each other's metadata. This approach proves particularly effective in cold start scenarios and demonstrates robust generalization capabilities. We employ a combination of both approaches in this work to generate enriched user and item information.

Parameter-Efficient Tuning for Personalization: Tan et al. [14] use low-rank adapters to create instruction-tuned LLMs for each user, enhancing accuracy and generalization of user behavior prediction. Our approach builds on this by incorporating comprehensive user profiles and enriched interactions to better encode preferences. Also, our aim is to check the effectiveness of the appraoch on SLMs. The minimal updated parameters and plug-and-play nature of PEFT make it ideal for

*Code at https://github.com/himansh005/persona_lora

efficient SLM personalization. Our work embeds user history within user level LoRA parameters, providing each user agent with a robust and expressive long-term memory.

LLMs for Personalization: Current research on LLM personalization focuses on crafting prompts using historical user data to customize responses [1]. Strategies include: (1) Vanilla Personalized Prompts: Use in-context and few-shot learning to embed user behavior history (e.g., personal rating histories by Dai et al. [2] and Kang et al. [5], task-specific interaction histories by Liu et al. [7]). (2) Retrieval-Augmented Personalized Prompts: Retrieve relevant user history to enhance prompts (e.g., LaMP [12], AuthorPred [6], Pearl [9]). (3) Profile-Augmented Personalized Prompts: Summarize user preferences into natural language profiles (e.g., Richardson et al. [11], ONCE [8]). These methods aim to improve the personalization of LLM responses. While these advancements are notable, the current methods struggle to effectively leverage long-term user preferences especially in SLMs. Our approach overcomes this limitation by embedding long-term preferences directly into the model parameters using the personal PEFT module.

3 Methodology

In this section, we define the standard notation and methodologies used throughout the paper. We begin by introducing the fundamental concepts and terminologies that form the basis of our approach. This includes the representation of users, items, and their interactions within a traditional recommender system. We then describe our approach on construction and training of user agents to predict user preferences.

3.1 Preliminaries

In a traditional recommender system, we have a set of users $U = \{u\}$, a set of items $I = \{i\}$, and a set of their interaction records denoted by $D = \{\langle u, i\rangle\}$. In our system, a user agent is built using an open source SLM and has short-term and long-term memories, denoted by M_s and M_l, respectively. Thus, a user agent can be defined as:

$$y_i \leftarrow f_{\text{SLM}}(M_s, \text{item}_i, \{\text{item}_1, \ldots, \text{item}_k\}),$$

where M_s is the user agent's short-term memory, item_i is the description of the i-th item, and $\{\text{item}_1, \ldots, \text{item}_k\} \subseteq M_l$ are items retrieved using item_i as a query by a similarity function $m(e(\text{item}_i), e(\text{item}_k)) < \delta$. We implement m as cosine similarity and e using a frozen embedding model, with δ being a threshold determined heuristically. Finally, y_i is the response of the SLM-based user agent and can have multiple tokens in its output.

3.2 Distilling User Preferences and Enriching Interactions

A key challenge of using SLMs for building user agents has been the appropriate representation of the user. Inspired by [13] and [17], we use an hierarchical self-reflection based algorithm that inputs a chronologically sorted user-item interaction dataset in batches D and outputs $D_{\text{enriched}} = \{\langle u_{\text{enriched}}, i_{\text{enriched}}\rangle\}$, where the user and item descriptions reflect the preferences of the user and the item, respectively. The process results in generation of these two key features per user:

User Profile (M_s): A description of general traits of the user inferred based on their interaction history. This act as the user's short term memory M_s.

Enriched User Interaction (M_l): A textual description that explains the why the user liked or disliked a given item. To create a persona agent with an SLM, we use M_s as a fixed system prompt during training and inference. We also use user profile embeddings to identify user clusters or personas. Additionally, M_l is embedded and stored in a database for use in RAG and RAFT. During training, we fetch the k nearest items using the current item embedding and use these as queries to retrieve their enriched versions.

3.3 Personalized Low-Rank Adapters

Central to our work is parameter efficiency, as we aim to simulate a very large number of users, which is typical in a recommender system. Previous research [14] has demonstrated that training multiple low-rank adapters per user can effectively capture user-specific knowledge and style. However, the number of parameters required for this approach would be enormous. Therefore, we propose that